Episodio 7 de 14: Rendimiento: quién gana, y el misterio de Folly

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 6, «Simular 4 procesadores con un solo PC».
- **Episodio siguiente:** episodio 8, «Latencia y memoria retenida».

Lo esencial de este episodio:

- Autores gana a Libstdc++ y a Williams. Anderson gana cuando hay muchas escrituras sobre una sola pila (de 1,87× a 1,99×). El mejor caso de Autores se reproduce casi exacto.
- Folly rinde aquí mucho mejor que en el paper, y la diferencia ya existe con un solo hilo: no es contención ni un fallo del montaje. La causa sigue abierta.
- Con un millón de pilas, cuatro algoritmos empatan entre 53 y 57 Mop/s. Los contadores muestran que tres llegan al 89-93 % del techo de la RAM de este PC, que tiene un solo módulo.

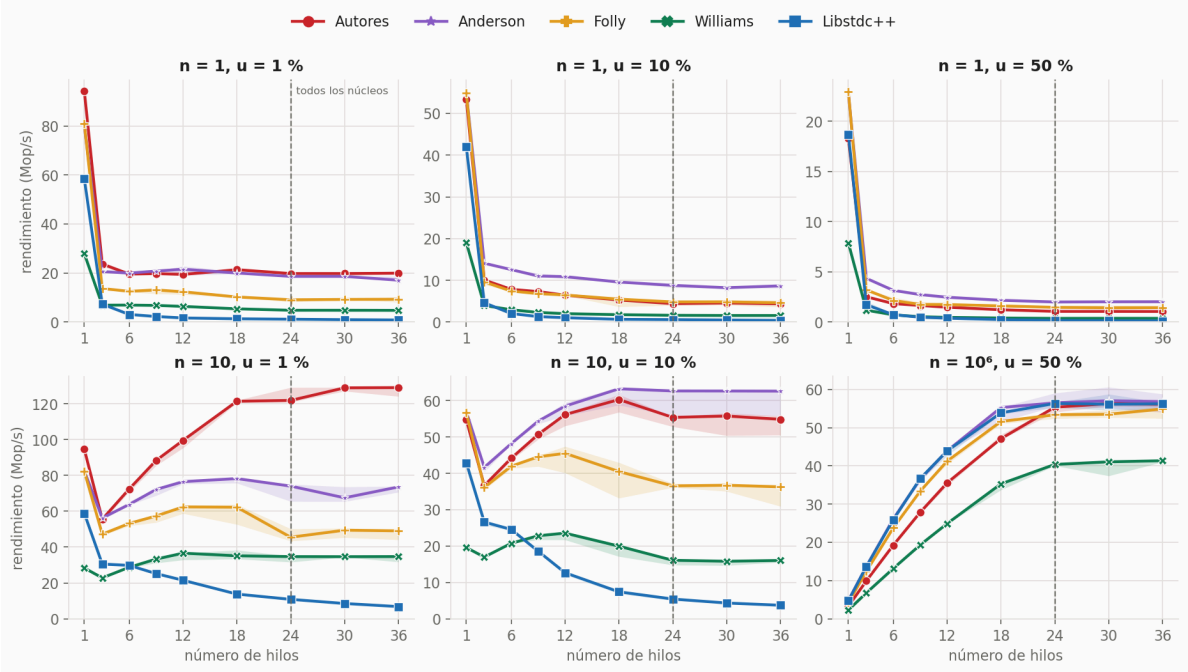
Parte 6. Los resultados, explicados

Tres reglas antes de mirar ninguna gráfica:

1. **No se comparan máquinas, se comparan algoritmos.** Esta CPU es casi diez años más moderna que los Xeon del paper, así que los números absolutos salen distintos y eso no significa nada. Lo que importa es el **orden** y la **proporción**: si un algoritmo era el doble de rápido que otro allí, ¿lo es también aquí?
2. **«0,50×» significa «rinde la mitad que Autores»**, y «2×», el doble. Es la forma de comparar sin que importe la máquina.
3. **Una diferencia pequeña no es una victoria.** Como verás en la Parte 6.7, repetir una medida con otra duración movió el resultado entre un 2 % y un 21 %. Por eso hablamos de **empate** cuando los rangos de las rondas se solapan o la diferencia es de pocas unidades por ciento.

Calidad de la tanda oficial en Docker: 2 700 ejecuciones de rendimiento, ninguna fallida; temperatura mediana 75 °C (solo 5 ejecuciones llegaron a 90 °C o más, con un máximo de 95 °C); la dispersión típica entre rondas fue del 1 %, y en los puntos más ruidosos, del 8 %.

6.1 Rendimiento: la figura 5



Rendimiento de las 5 implementaciones en los 6 escenarios

Cómo leerla. Cada recuadro es un escenario. Recuerda: **n** es el número de pilas (pocas significa mucha gente en poco sitio) y **u**, el porcentaje de escrituras. Hacia la derecha hay más hilos; hacia arriba, más trabajo por segundo. La línea es la mediana de 5 rondas, y la sombra va de la peor ronda a la mejor. La raya discontinua marca los 24 núcleos: a su derecha hay más hilos que núcleos, igual que en el paper a partir de 144.

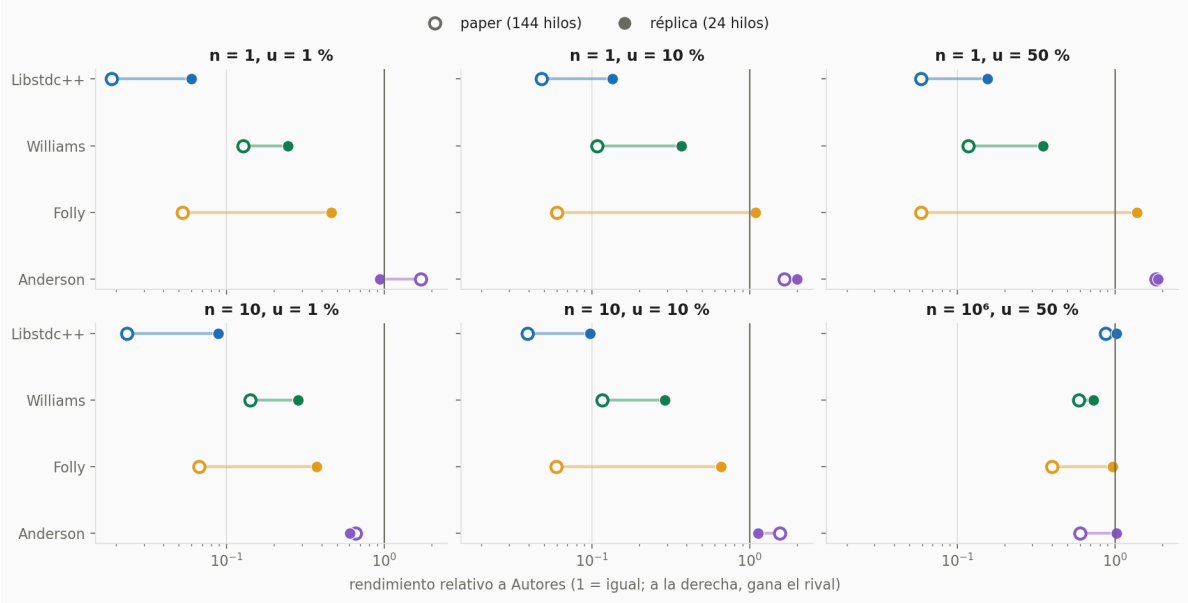
Lo primero que salta a la vista: de 1 a 3 hilos casi todo se desploma. Con n = 1 y u = 1 %, Autores pasa de 94 a 24 Mop/s. Parece un fallo y no lo es. Es el cuaderno de la Parte 2.3: con un hilo, el cuaderno nunca sale de tu mano; con tres, se pasa el tiempo viajando. Añadir gente a un único cuaderno no acelera nada, lo empeora.

Mira ahora el recuadro de n = 10^6. Ahí cada hilo trabaja en su rincón, casi nunca coincide con otro, y las curvas **suben**. Eso es lo que uno espera de «más hilos, más trabajo». La diferencia entre un recuadro y otro tiene un nombre: **contención**.

El marcador, con toda la máquina en marcha:

Escenario	En palabras	Paper (144 hilos)	Réplica (24 hilos)	¿Coincide quién gana?
n = 1, u = 1 %	una pila, casi todo lecturas	Anderson (1,70×)	Autores, por poco (Anderson 0,94×)	No
n = 1, u = 10 %	una pila, algunas escrituras	Anderson (1,66×)	Anderson (1,99×)	Sí
n = 1, u = 50 %	una pila, muchas escrituras	Anderson (1,82×)	Anderson (1,87×)	Sí
n = 10, u = 1 %	10 pilas, casi todo lecturas	Autores (Anderson 0,66×)	Autores (Anderson 0,61×)	Sí, casi exacto
n = 10, u = 10 %	10 pilas, algunas escrituras	Anderson (1,54×)	Anderson (1,13×)	Sí, por menos
n = 10^6, u = 50 %	un millón de pilas	Autores	empate a cuatro, unos 55 Mop/s	No

Coincide en 4 de 6. La gráfica siguiente enseña lo mismo para todos los rivales a la vez:



Cuánto rinde cada rival respecto a Autores, en el paper y en la réplica

Cómo leerla. Cada fila es un rival. El círculo **hueco** es el paper y el **relleno**, la réplica. La raya vertical es Autores: a su izquierda el rival pierde, a su derecha gana. La escala es logarítmica: de 10^{-1} a 10^{0} hay un factor 10. Si hueco y relleno están juntos, la réplica clavó al paper.

Lo que se repite

- **Libstdc++ se hunde al crecer los hilos.** Pasa de 58,5 a 1,2 Mop/s con n = 1 y u = 1 %: unas 49 veces menos. En el paper, de 7 a 0,11: unas 64 veces menos. Es lo que hace un candado: formar una cola.
- **Williams queda por debajo de Autores en los 6 escenarios**, entre 0,24× y 0,73×. Dos bucles de `DWCAS` por lectura se pagan siempre.
- **Anderson gana cuando hay escrituras sobre pocas pilas.** Es justo lo que explica el paper: en Autores cada lectura modifica el tablón, y eso hace fallar los `CAS` de quien quiere escribir. En Anderson leer no toca el tablón.
- **El mejor caso de Autores se reproduce casi exacto.** Con n = 10 y u = 1 %: Autores 122 Mop/s, Anderson 74, Folly 46, Williams 35, Libstdc++ 11. Pocas escrituras y algo de reparto: el terreno donde un único `fetch_sub` que nunca falla brilla.

Lo que no se repite, y qué sabemos del porqué

1. Folly rinde muchísimo mejor aquí. Con contención, en el paper rinde entre 0,05× y 0,07× de Autores. Aquí, entre 0,37× y 1,37×, y con n = 1 y u = 50 % le gana (1,47 frente a 1,07 Mop/s, sin que los rangos se toquen).

¿Será cosa de Docker, o de haber medido 2 segundos en vez de 10? **No.** Las dos cosas se comprobaron:

- **Sin Docker**, con los mismos programas, sale casi igual: entre 0,37× y 1,40× con 24 hilos. Y Folly vuelve a ganar con n = 1 y u = 50 % (1,50 frente a 1,07 Mop/s).
- **Con la receta exacta del paper**, 7 ejecuciones de 10 s, también: entre 0,36× y 1,26×.

Es decir, ni el envoltorio ni el recorte explican la diferencia. Los datos están en las tablas «Validación de los recortes (nativo_oficial)» y «Docker frente a nativo (binarios oficiales)» de `tablas.md`.

La tentación es decir «claro, el paper tiene 4 sockets y aquí hay uno». **Esa explicación no se sostiene**, y la razón es instructiva. Mira qué pasa con **un solo hilo**, donde no hay pelea ni nada viaja entre núcleos:

Escenario, con 1 hilo	Folly en el paper	Folly aquí	Folly aquí, sin Docker	Williams en el paper	Williams aquí
n = 1, u = 1 %	0,20×	0,86×	0,88×	0,36×	0,30×
n = 10, u = 10 %	0,21×	1,04×	1,04×	0,38×	0,36×
n = 10^6, u = 50 %	0,46×	1,29×	1,24×	0,68×	0,70×

La brecha de Folly **ya está ahí con un hilo**, mientras que Williams coincide bastante. Así que la causa no es la pelea entre hilos ni el viaje de líneas entre ellos. Hay además una pista en el propio paper: en su figura 6, medida en otra máquina más moderna, el `load` de Folly sin contención tarda 24 ns, casi lo mismo que Autores (23 ns). Folly no es lento de nacimiento: algo de la máquina de la figura 5, o de cómo se midió allí, le penaliza. Podría ser la generación del chip o que la memoria estaba repartida entre los 4 sockets (`numactl -i all`) incluso con un hilo. **Con este PC no se puede saber.** Queda como pregunta abierta.

2. Con n = 1 y u = 1 %, Anderson no adelanta a Autores. Aquí hay un matiz importante. En el paper Anderson solo adelanta a Autores **a partir de 54 hilos**: con 18 rinde 0,72× y con 36, 0,89×. Aquí, con 18 hilos, rinde 0,94×. A igual número de hilos, los dos resultados no están tan lejos: esta máquina no llega a la zona donde ocurre el cruce. Parece una diferencia de alcance más que una contradicción.

3. Con un millón de pilas, todo empata. En el paper Autores lidera con 129 Mop/s. Aquí Autores, Anderson, Libstdc++ y Folly se quedan entre 53 y 57 Mop/s. Que cuatro algoritmos tan distintos toquen el mismo techo sugiere que el límite no está en el algoritmo, sino en algo común: con un millón de pilas casi cada operación tiene que ir a la RAM, y el paper tenía 4 chips con su memoria cada uno, frente a uno solo aquí.

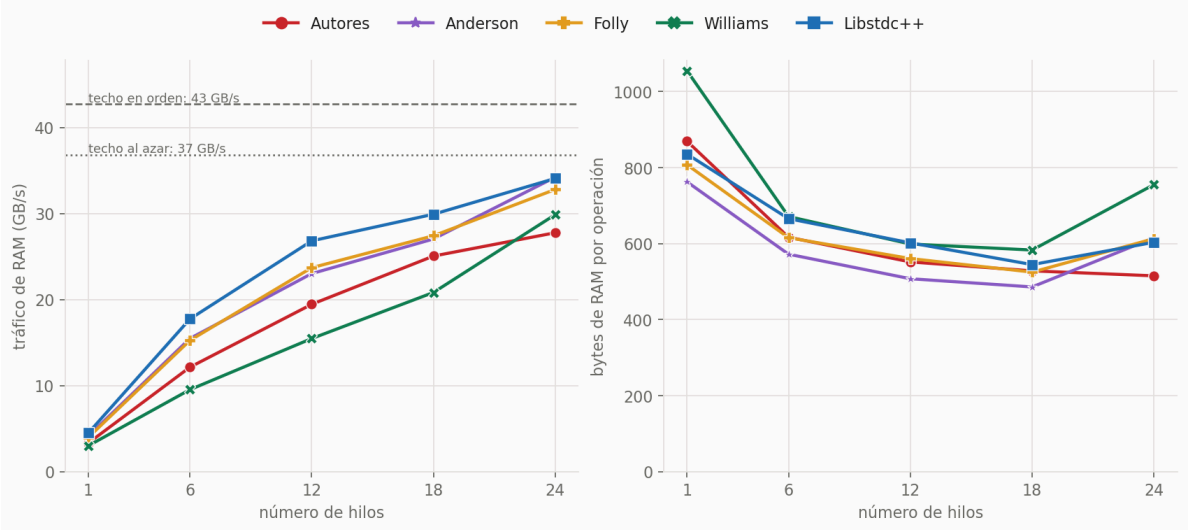
Eso se comprobó midiendo. La CPU trae unos contadores en su **controladora de memoria**, la pieza por la que pasa todo lo que entra y sale de la RAM. Son como un cuentakilómetros en la única carretera que lleva a un almacén: no hay que adivinar cuánto tráfico hay, basta con leerlo. Lo lee `herramientas/medir_memoria.py`.

Y esa carretera aquí es estrecha. Este PC tiene **un solo módulo de RAM**, con la otra ranura vacía: un solo canal, con 44,8 GB/s como máximo teórico. Hay dos techos reales, porque no es lo mismo leer en orden que saltar de un sitio a otro. Leer un libro página a página es más rápido que abrirlo en páginas al azar:

Techo medido	GB/s
Leyendo en orden	43
Leyendo líneas sueltas al azar, que es como acceden las pilas	37

Con 24 hilos, n = 10^6 y u = 50 %:

Algoritmo	Mop/s	Tráfico de RAM (GB/s)	Parte del techo al azar	Bytes de RAM por operación
Anderson	54,5	34,2	93 %	615
Libstdc++	55,3	34,1	92 %	603
Folly	52,4	32,8	89 %	613
Williams	38,6	29,9	81 %	755
Autores	52,5	27,8	75 %	515



Tráfico real de RAM

Qué se saca de aquí:

- **La explicación de la memoria sale reforzada.** Anderson, Libstdc++ y Folly rozan el techo y casi dejan de crecer: de 18 a 24 hilos, Anderson pasa de 54,2 a 54,5 Mop/s mientras su tráfico sube de 27 a 34 GB/s. Con 36 hilos, ninguno pasa de 57 Mop/s.
- **Pero no lo explica todo.** Autores rinde igual con solo un 75 % del techo, y con 30 y 36 hilos tampoco lo supera. Algo más lo frena, quizá el tiempo de espera de cada acceso. Eso no se midió.
- **No está demostrado.** El techo al azar solo lee, y las pilas leen y escriben, así que su techo real podría ser algo distinto. La prueba definitiva sería repetirlo con dos módulos: si el rendimiento sube, queda demostrado.
- **Autores gasta menos.** Mueve un 15-32 % menos de bytes por operación que los demás con 24 hilos. Ese ahorro podría notarse en una máquina donde la RAM frene más, pero aquí no se comprobó.

Y hay un detalle incómodo. Por debajo de ese techo, con pocos hilos, Autores va **por detrás**: con 1 hilo Anderson rinde 1,46× y Libstdc++ 1,44×; con 12 hilos, 1,24× los dos. En el paper, con 1 hilo, Anderson va 1,03× y Libstdc++ 0,87×. Mover más bytes no lo explica: con 6 hilos, Autores mueve 617 bytes por operación, Anderson 572 y Libstdc++ 666. Libstdc++ mueve **más** que Autores y aun así va 1,38× más rápido. La causa sigue sin aclararse. Sin Docker, con los mismos programas, se repite: con 1 hilo, Anderson rinde 1,33× y Libstdc++ 1,40×; con 12 hilos, 1,19× los dos. Así que tampoco es cosa de Docker.

¿Y el número τ de `tablas.md`? La τ de Kendall mide si dos listas están en el mismo orden: +1 es idéntico, 0 es sin relación, −1 es al revés. Aquí vale +0,60, +0,60, +0,40, +0,80, +0,80 y +0,20. Siempre positiva: el orden siempre se parece. En los cinco primeros escenarios, casi todo el desacuerdo viene de Folly; el +0,20 del último es el empate, donde el orden entre iguales no significa nada.